

ion.

value into a
section 60

nodes

ation.

sides

Analytical Problem Solving:

1. Perform the following operations in LIFO data structures having an array size of 5: Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(), Pop(). Print the peak elements of data structure and its index position with neat sketch explained in detail.

Amal
29/7

Problem Statement:

Perform the following operations on a stack of size 5:

Operations:

Push(45), Push(22), Push(11), Push(77), Pop(), Push(44), Push(33), Pop(), Pop(), Pop().

After performing all the operations, print the peak (top) element and its index position.

Introduction:

A stack is a linear data structure that follows LIFO principle. In this principle, the element inserted last is the first element to be removed. A stack can be implemented using an array or a linked list. In an array implementation, a variable called top keeps track of the most element in the stack.

The two basic operations of a stack are:

- * Push: Inserts an element at top of the stack.
- * Pop: Removes the top element from the stack.

The stack also supports the peek (or top) operation, which returns the top element without removing it.

Algorithm:

Push Operation:

1. Check whether the stack is full.
2. If the stack is full, display Stack Overflow.
3. Otherwise, increment the TOP by 1.
4. Insert the new element at the TOP position.

POP Operation:

1. Check whether the stack is empty.
2. If the stack is empty, display Stack Underflow.
3. Otherwise, remove element at the TOP position.
4. Decrement the TOP by 1.

Execution of Operations:

Steps	Operation	Stack (Bottom → top)
1	Push(45)	45
2	Push(88)	45, 88
3	Push(11)	45, 88, 11
4	Push(77)	45, 88, 11

5	POP()	45, 88, 11
6	PUSH(44)	45, 88, 11, 44
7	PUSH(33)	45, 88, 11, 44, 33
8	POP()	45, 88, 11, 44
9	POP()	45, 88, 11
10	POP()	45, 88

Final Stack:

TOP
 ↓
 +-----+
 | 88 | Index = 1
 +-----+
 | 45 | Index = 0
 +-----+

Analysis:

The stack follows LIFO Principle. During the execution:

1. Five Push operations inserted elements into stack.
2. Four pop operations removed the most recently inserted elements.
3. The last 3 removed elements were 33, 44 and 11, leaving 45 and 88 in the stack.
4. The Topmost element after all operations is 88, stored at index position 1.

Applications of Stack:

1. Function call management in programming
2. Expression evaluation and conversion.
3. Parentheses matching.
4. Undo and Redo operations in editors.
5. Browser history navigation.

Advantages of Stack:

1. Easy to implement
2. Fast insertion and deletion.
3. Efficient memory usage
4. Suitable for recursive function calls.

Conclusion:

A Stack is one of the most important linear data structures used in Computer Science. It follows the LIFO principle, where the most recently inserted element is removed first. After performing all given operations, the stack contains 45 and 88. Therefore, the peak (top) element is 88, and its index position is 1. This example clearly demonstrates the working of Push, Pop, and Peek operations in a stack implemented using an array.

How do you implement Queue data Structure using Single Linked List? with a neat diagram discussing Enqueue and Dequeue operations using SL.

Problem Statement:

Explain how to implement a Queue using a Single Linked List (SL). Discuss the Enqueue and Dequeue operations with suitable diagrams.

Introduction:

A Queue is a Linear data Structure that follows the FIFO principle. The first element inserted into the queue is the first one removed. A queue can be implemented using a Single Linked List, where each node contains a data field and a pointer to the next node. Two pointers, Front and Rear, are used to manage the queue.

Node Structure:

Struct Node

```
{  
    int data;  
    Struct Node* next;  
}
```

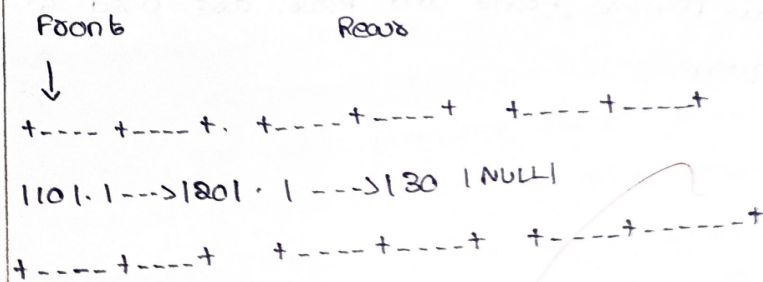
Enqueue Operation:

The Enqueue operation inserts a new node at the rear of the queue.

Algorithm:

1. Create a new node.
2. Store the data in the node.
3. Set the next pointer of the new node to NULL.
4. If the queue is empty, assign both Front and Rear to the new node.
5. Otherwise, connect the new node to the Rear node.
6. Update Rear to the new node.

Diagram:



Dequeue Operation:

The Dequeue operation removes the node from Front of the queue.

Algorithm:

1. check whether the queue is empty.
2. If empty, report Queue Underflow.
3. Store the Front node in a temporary pointer.
4. move Front to the next node.
5. Delete the temporary node.
6. If Front becomes NULL. set Rear to NULL.

Diagram:

Before Dequeue

Front



10 → 20 → 30 → NULL



Rear

After Dequeue:

Front



20 → 30 → NULL



Rear

Analysis:

The Queue follows FIFO Principle, ensuring that elements are removed in the same order in which they were inserted. The Dequeue operation removes from the Front. Both operations are performed efficiently in O(1) time.

Advantages:

1. Dynamic memory allocation.
2. No Fixed Size limitation.
3. Efficient insertion and deletion.
4. Better memory utilisation.
5. Suitable for real-time applications.

Applications

1. CPU Scheduling.
2. Printer Spooling.
3. Ticket Reservation Systems.
4. Breadth First Search (BFS).
5. Network Packet Processing.

Conclusion:

Implementing a Queue using a Single Linked List (SL) provides flexibility and efficient memory usage. By maintaining Front and Rear pointers, both insertion and deletion are performed quickly, making this implementation suitable for operating systems, networking and process scheduling applications.